Operating System Project
Phase#2

Memory Management

Introduction:

Memory management is a fundamental responsibility of any modern operating system. As processes compete for limited physical memory, the OS must provide mechanisms that ensure efficiency, fairness, isolation, and high system performance. Paging is one of the most widely used memory management techniques, offering a practical solution for eliminating external fragmentation while simplifying memory allocation.

In this project, you will explore how paging work. You will model the structure of virtual and physical address spaces, design and manage page tables, simulate address translation, and evaluate system behaviour under various paging-related scenarios.

Objectives:

By completing this project, students should be able to:

- Understand Paging Fundamentals
- Implement Core Paging Structures
- Apply Memory Allocation and Management Techniques
- Develop Debugging and Verification Skills on linux.

System Description & Assumptions:

You have a single CPU with 10-bit byte addressable memory (each address access one byte). The memory (RAM) has 512 bytes. Page size is 16 bytes. Memory access takes one cycle (instantly), while disk access takes 10 cycles.

Second Chance algorithm is used as the primary swapping algorithm. Assume that pages required by OS itself is omitted from the simulation for simplicity.

Requirement

You are required to edit the *Scheduler* (in code file *scheduler.c*) from the "OS Scheduler" and add **MMU.c** to include the basic functions for memory paging allocation capabilities. It should **allocate** pages for processes as they **request** or **swap** them when needed and free them as they leave so that it can be re-used by later processes. For simplicity, integrate this module with **Round Robin**

- First, you will need to define a structure for your physical pages, to know which pages are empty (Free-linked list or Free bit map or make your own structure).
- Secondly, you need to create the appropriate data structure to perform page swapping using second change algorithm, you should keep also which process each page belongs too.
- Thirdly, you should decide the page table entry size and content. (Hint: we don't store Virtual Frame, just physical and whatever flags you need)
- Whenever the OS demands a page, we first search the free pages before swapping.
- When process first start,
    - It should allocate a free Page for its page table (initially all entries are invalid).
    - If no Free pages available, use Second Chance algorithm to allocate the page table.
    - You should update the PCB by the address of the process page table (take care, each process has its own page table).
    - Load the first page of the process.
    - Update the Page table of the requesting process.
    - **For Simplicity**, you can assume that we only need first page of page table and that the allocation and first initial loading take no time
    - **For Simplicity**, **Don't Swap out Process Tables.**
- When Page Fault occurs, Demand a page, update the Process Table accordingly and block the process.
- The page is requested by the process at specific instances during its runtime.

- Take care a **modified page** should be written to disk (file) before being swapped. (we will access disk twice).
- Initially assume memory is all free.

- **Bonus**:
    - USE LRU algorithm to make swapping
    - Use multi-level page table.
    - Use Multi-level scheduling

Input:

You will need to (slightly) modify the *Process Generator* (in code file *process generator.c*) to accept an extra process information; *and the base + limit address of the process on the disk. (For simplicity, assume base is the page number on disk and Limit is the number of pages required by the process)*

processes.txt example

#id	arrival	runtime	priority	dependencyId	base	Limit
1	1	6	5	-1	0	1000
2	3	3	3	-1	4000	2000

- Comments are added as lines beginning with # and should be ignored.
- Each non-comment line represents a process.
- Fields are separated with *one tab character '\t'*.
- You can assume that processes are sorted by their arrival time. *Take care that 2 or more processes may arrive at the same time.*

You will also have per process file representing the addresses requested, the time in the process file is with respect to its start time, not absolute. For example if a process starts at 8, time:2 means 10.

requests.txt example

#time	#addressInBinary	#r/w
1	0	r
2	1	r
3	100	w

Output

A new output file *memory.log* should be generated for the memory information.

memory.log example

#PageFault upon VA "xx" from process "yy"
#Free Physical page "ZZ" allocated
#Swapping out page "ZZ" to disk
#At time "i" disk address "X" for process "y" is loaded into memory page "ZZ" .

in case of no free page and no swapping back to disk
PageFault upon VA 100110 from process 1
At time 1 page 1 for process 1 is loaded into memory page 10.

in case of free page
PageFault upon VA 10 from process 2
Free Physical page 2 allocated
At time 1 page 2 for process 2 is loaded into memory page 2.

- Comments and empty lines are added as lines beginning with # and should be ignored.
- You need to stick to the given format because files are compared automatically.

Guidelines:

- Read the document carefully at least once.
- Your program must not crash.
- Spend a good time in design, and it will make your life much easier in implementation.
- The code should be clearly commented, and the variable names should be indicative.
- Correct any mistakes discussed during phase 1.
- Violation of Academic integrity leads to negative project grade.

Deliverables

You should deliver a public repo including the full project along with a final report that includes any information about the system, including the system block diagram, process states, and all the necessary information

Grading:

- Correct Data structures, page table structure, and logic (20%).
- Correct swapping handling (10%).
- Correct process blocking during swap (10%).
- Correct integration with RR (20%).
- Correct Second Chance implementation(20%).
- Integration and correct format (20%).
- Plagiarism (-100%).